COMPUTER ENGINEERING

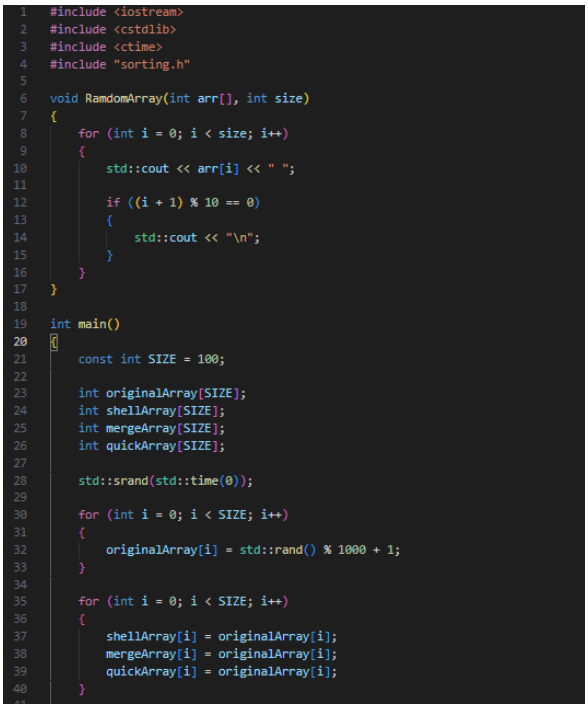
QUEZON CITY

COURSE CODE/TITLE:	CPE-010/Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	09/13/2026	**DATE SUBMITTED:**	09/16/2026
NAME:	Almirol Joel A	**INSTRUCTOR:**	Engr.Jimlord Quejado

ACTIVITY NO. 8.1 Sorting Algorithms Pt2

1. PROCEDURE OUTPUT

What procedures did you perform?

Code + Console Screenshot	

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

void RamdomArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        std::cout << arr[i] << " ";

        if ((i + 1) % 10 == 0)
        {
            std::cout << "\n";
        }
    }
}

int main()
{
    const int SIZE = 100;

    int originalArray[SIZE];
    int shellArray[SIZE];
    int mergeArray[SIZE];
    int quickArray[SIZE];

    std::srand(std::time(0));

    for (int i = 0; i < SIZE; i++)
    {
        originalArray[i] = std::rand() % 1000 + 1;
    }

    for (int i = 0; i < SIZE; i++)
    {
        shellArray[i] = originalArray[i];
        mergeArray[i] = originalArray[i];
        quickArray[i] = originalArray[i];
    }
```

```
    std::cout << "Original Array:\n\n";
    RamdomArray(originalArray, SIZE);

    shellSort(shellArray, SIZE);

    std::cout << "\nShell Sort:\n\n";
    RamdomArray(shellArray, SIZE);

    mergeSort(mergeArray, 0, SIZE - 1);

    std::cout << "\nMerge Sort:\n\n";
    RamdomArray(mergeArray, SIZE);

    quickSort(quickArray, 0, SIZE - 1);

    std::cout << "\nQuick Sort:\n\n";
    RamdomArray(quickArray, SIZE);

    return 0;
}
```

```
Original Array:

667 698 932 106 850 876 789 86 218 592
9 820 325 693 465 916 827 522 122 64
927 152 489 380 858 326 67 943 188 356
588 937 127 499 830 157 631 155 732 737
863 548 455 867 350 393 929 215 842 209
936 102 771 632 45 633 843 454 582 350
965 518 881 866 516 666 402 856 293 157
327 672 719 73 825 244 217 978 790 64
509 651 509 632 141 535 473 837 51 747
322 51 29 258 440 141 952 800 810 914
```

Observations	The 100-element integer array is randomly generated with pseudo-random numbers between 1 and 1000 using std::rand() % 1000 + 1. The initial array is displayed in a 10x10 table format on the console screen to form the unsorted array as the basis for duplicating into shellArray, mergeArray, and quickArray.

Table 8-1. Array of Values for Sort Algorithm Testing

Code + Console Screenshot

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

void RamdomArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        std::cout << arr[i] << " ";

        if ((i + 1) % 10 == 0)
        {
            std::cout << "\n";
        }
    }
}

int main()
{
    const int SIZE = 100;

    int originalArray[SIZE];
    int shellArray[SIZE];
    int mergeArray[SIZE];
    int quickArray[SIZE];

    std::srand(std::time(0));

    for (int i = 0; i < SIZE; i++)
    {
        originalArray[i] = std::rand() % 1000 + 1;
    }

    for (int i = 0; i < SIZE; i++)
    {
        shellArray[i] = originalArray[i];
        mergeArray[i] = originalArray[i];
        quickArray[i] = originalArray[i];
    }
```

```cpp
    std::cout << "Original Array:\n\n";
    RamdomArray(originalArray, SIZE);

    shellSort(shellArray, SIZE);

    std::cout << "\nShell Sort:\n\n";
    RamdomArray(shellArray, SIZE);

    mergeSort(mergeArray, 0, SIZE - 1);

    std::cout << "\nMerge Sort:\n\n";
    RamdomArray(mergeArray, SIZE);

    quickSort(quickArray, 0, SIZE - 1);

    std::cout << "\nQuick Sort:\n\n";
    RamdomArray(quickArray, SIZE);

    return 0;
}
```

```
Shell Sort:

9 29 45 51 51 64 64 67 73 86
102 106 122 127 141 141 152 155 157 157
188 209 215 217 218 244 258 293 322 325
326 327 350 350 356 380 393 402 440 454
455 465 473 489 499 509 509 516 518 522
535 548 582 588 592 631 632 632 633 651
666 667 672 693 698 719 732 737 747 771
789 790 800 810 820 825 827 830 837 842
843 850 856 858 863 866 867 876 881 914
916 927 929 932 936 937 943 952 965 978
```

Observations	Shows time complexity of O(n log n) to O(n^1.5) through the comparison of distant elements with diminishing gap values. After running, the baseline input array undergoes transformation to become sorted in an increasing numerical order.

Table 8-2. Shell Sort Technique

Code + Console Screenshot

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

void RamdomArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        std::cout << arr[i] << " ";

        if ((i + 1) % 10 == 0)
        {
            std::cout << "\n";
        }
    }
}

int main()
{
    const int SIZE = 100;

    int originalArray[SIZE];
    int shellArray[SIZE];
    int mergeArray[SIZE];
    int quickArray[SIZE];

    std::srand(std::time(0));

    for (int i = 0; i < SIZE; i++)
    {
        originalArray[i] = std::rand() % 1000 + 1;
    }

    for (int i = 0; i < SIZE; i++)
    {
        shellArray[i] = originalArray[i];
        mergeArray[i] = originalArray[i];
        quickArray[i] = originalArray[i];
    }
```

```cpp
    std::cout << "Original Array:\n\n";
    RamdomArray(originalArray, SIZE);

    shellSort(shellArray, SIZE);

    std::cout << "\nShell Sort:\n\n";
    RamdomArray(shellArray, SIZE);

    mergeSort(mergeArray, 0, SIZE - 1);

    std::cout << "\nMerge Sort:\n\n";
    RamdomArray(mergeArray, SIZE);

    quickSort(quickArray, 0, SIZE - 1);

    std::cout << "\nQuick Sort:\n\n";
    RamdomArray(quickArray, SIZE);

    return 0;
}
```

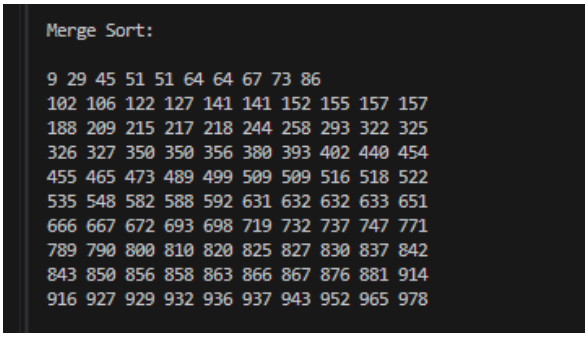
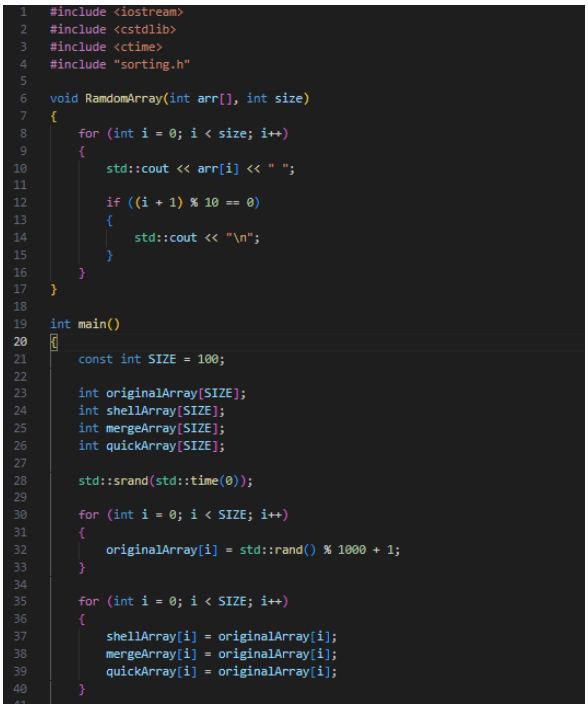
	
Observations	It works based on the principle of divide and conquer where the array is divided into two parts until arrays containing one element each are left, and these are then merged to give an array which is sorted. It gives a totally sorted array but requires space.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot	

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "sorting.h"

void RamdomArray(int arr[], int size)
{
    for (int i = 0; i < size; i++)
    {
        std::cout << arr[i] << " ";

        if ((i + 1) % 10 == 0)
        {
            std::cout << "\n";
        }
    }
}

int main()
{
    const int SIZE = 100;

    int originalArray[SIZE];
    int shellArray[SIZE];
    int mergeArray[SIZE];
    int quickArray[SIZE];

    std::srand(std::time(0));

    for (int i = 0; i < SIZE; i++)
    {
        originalArray[i] = std::rand() % 1000 + 1;
    }

    for (int i = 0; i < SIZE; i++)
    {
        shellArray[i] = originalArray[i];
        mergeArray[i] = originalArray[i];
        quickArray[i] = originalArray[i];
    }
```

```
42      std::cout << "Original Array:\n\n";
43      RamdomArray(originalArray, SIZE);
44
45      shellSort(shellArray, SIZE);
46
47      std::cout << "\nShell Sort:\n\n";
48      RamdomArray(shellArray, SIZE);
49
50      mergeSort(mergeArray, 0, SIZE - 1);
51
52      std::cout << "\nMerge Sort:\n\n";
53      RamdomArray(mergeArray, SIZE);
54
55      quickSort(quickArray, 0, SIZE - 1);
56
57      std::cout << "\nQuick Sort:\n\n";
58      RamdomArray(quickArray, SIZE);
59
60      return 0;
61  }
```

```
Quick Sort:

9 29 45 51 51 64 64 67 73 86
102 106 122 127 141 141 152 155 157 157
188 209 215 217 218 244 258 293 322 325
326 327 350 350 356 380 393 402 440 454
455 465 473 489 499 509 509 516 518 522
535 548 582 588 592 631 632 632 633 651
666 667 672 693 698 719 732 737 747 771
789 790 800 810 820 825 827 830 837 842
843 850 856 858 863 866 867 876 881 914
916 927 929 932 936 937 943 952 965 978
PS C:\Users\TIPQC\Documents\almirol 8.1 sorting>
```

Observations	Makes use of dynamic partitioning through a pivot element to arrange elements in sub-arrays that contain lower and higher elements respectively. Gives a sorted sequence in ascending order with high efficiency (time complexity O(n \log n)) and small memory usage (stack space O(\log n)).

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.

ANSWER:

Yes, the sub-lists (left and right) can indeed be sorted using different sorting algorithms. After the placement of the pivot at the correct index by the Quick Sort partitioning process, the elements on the left and right form separate sub-arrays. Although the sub-arrays formed are mostly sorted using the same Quick Sort algorithm again, the use of any algorithm (such as Insertion Sort, Selection Sort, and Merge Sort) is possible in these sub-arrays. Actually, the

use of hybrid sorting algorithms like IntroSort uses Quick Sort until the size of the sub-arrays becomes small (less than 16), where it switches to Insertion Sort.

Demonstrate an example.

Initial Array: [48, 12, 87, 34, 4]

Partition Step (Pivot = 34):

- Reordering places elements smaller than 34 on the left and larger on the right:
- Left sub-list: [12, 4]
- Pivot in place: [34]
- Right sub-list: [48, 87]

Sorting Sub-lists with Insertion Sort:

- Sort left sub-list [12, 4] using Insertion Sort \rightarrow [4, 12]
- Sort right sub-list [48, 87] using Insertion Sort \rightarrow [48, 87]

Combined Result: [4, 12, 34, 48, 87]

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N • log N) for their time complexity?

ANSWER:

The Fastest Algorithm for the Provided Data Set: Insertion Sort or Quick Sort/Merge Sort. In the case of a very small data set that consists of 15 different values, the fastest execution time is achieved by the Insertion Sort algorithm, due to the reduced overhead of function calls and efficient caching, even though it has a quadratic complexity in its worst-case scenario. Nevertheless, the fastest execution time from a theoretical point of view is achieved by Quick/Merge Sort.

Divide Step (\log N levels): Both algorithms repeatedly split the array in half (or into partitions). Halving an array of size N recursively creates a tree structure with a height/depth of O(\log N)

Combine/Partition Step (N work per level): At each level of the recursion tree, total processing time across all sub-problems takes O(N) operations (merging two sorted arrays in Merge Sort, or partitioning around a pivot in Quick Sort).

Total Time: Multiplying the work per level by the number of levels yields O(N \cdot \log N).

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.